C PROGRAMMING | ANSWER KEY

Structures & Unions in C

Solutions with sample output and explanations

22	5	20
Solutions	Sections	Programs tested

Contents

A	**Structure Basics** Declaring, reading, printing and copying structures.	A1 - A5
B	**Arrays of Structures and Functions** Handling many records and passing structures to functions.	B1 - B5
C	**Nested Structures and Pointers** A structure inside a structure, and the arrow operator.	C1 - C4
D	**Unions** Shared memory, sizeof, and when to use a union.	D1 - D5
E	**Mixed Practice** Spot the errors, then put it all together.	E1 - E3

How to use this key

1	**Try first.** Attempt the question yourself, then compare with the solution.
2	**Your answer may differ and still be right.** Different variable names, loops or layouts are fine.
3	**Outputs are real.** Each program was compiled and run with the sample input shown.
4	**Some values depend on the compiler.** `sizeof` results (A5, D2) and the number in D1 can differ on other systems. The reasoning matters more than the exact number.

Programs read names as a single word (no spaces) to keep `scanf` simple.

A Structure Basics

Declaring, reading, printing and copying structures.

A1 Student Details — EASY

```c
#include <stdio.h>

struct Student { int roll; char name[30]; float marks; };

int main(void) {
    struct Student s;
    printf("Enter roll, name and marks:\n");
    scanf("%d %s %f", &s.roll, s.name, &s.marks);

    printf("Roll: %d  Name: %s  Marks: %.2f\n", s.roll, s.name, s.marks);
    if (s.marks >= 40)
        printf("Pass\n");
    else
        printf("Fail\n");
    return 0;
}
```

Sample input: `1 Ravi 78.5`

OUTPUT

```
Enter roll, name and marks:
Roll: 1  Name: Ravi  Marks: 78.50
Pass
```

WHY

- Members are reached with the dot operator: `s.roll`, `s.name`, `s.marks`.
- `name` is a character array, so `scanf` needs no `&` for it. The other two need `&`.

A2 Book with Discount

EASY

```c
#include <stdio.h>

struct Book { char title[30]; char author[30]; float price; };

int main(void) {
    struct Book b1, b2;
    float newPrice;

    printf("Enter title, author and price of book 1:\n");
    scanf("%s %s %f", b1.title, b1.author, &b1.price);
    printf("Title: %s  Author: %s  Price: %.2f\n", b1.title, b1.author, b1.price);

    newPrice = b1.price - b1.price * 10 / 100;
    printf("Price after 10%% discount: %.2f\n", newPrice);

    printf("Enter title, author and price of book 2:\n");
    scanf("%s %s %f", b2.title, b2.author, &b2.price);

    if (b1.price > b2.price)
        printf("Costlier book: %s\n", b1.title);
    else
        printf("Costlier book: %s\n", b2.title);
    return 0;
}
```

Sample input: `Wings Kalam 250 | Gita Tagore 300`

OUTPUT

```
Enter title, author and price of book 1:
Title: Wings  Author: Kalam  Price: 250.00
Price after 10% discount: 225.00
Enter title, author and price of book 2:
Costlier book: Gita
```

WHY

- Discount: new price = price - price * 10 / 100 (same as `price * 0.9`).
- Two separate variables `b1` and `b2` are compared with `b1.price > b2.price`.
- Use `%%` inside `printf` to print a percent sign.

A3 Initialising and Copying

EASY

```c
#include <stdio.h>

struct Point { int x, y; };

int main(void) {
    struct Point p1 = {3, 4};
    struct Point p2 = p1;      /* copy all members */

    p2.x = 10;                 /* only p2 changes */

    printf("p1 = (%d, %d)  sum = %d\n", p1.x, p1.y, p1.x + p1.y);
    printf("p2 = (%d, %d)  sum = %d\n", p2.x, p2.y, p2.x + p2.y);
    return 0;
}
```

OUTPUT

```
p1 = (3, 4)  sum = 7
p2 = (10, 4)  sum = 14
```

WHY

- `p2 = p1;` copies every member. After that the two variables are independent.
- So changing `p2.x` to 10 does **not** change `p1`; `p1` is still (3, 4).

A4 Predict the Output

EASY

```c
#include <stdio.h>

struct Item { int qty; float price; };

int main(void) {
    struct Item a = {5, 20.5};
    struct Item b = a;
    b.qty = 10;
    printf("%d %.1f\n", a.qty, a.price);
    printf("%d %.1f\n", b.qty, b.price);
    return 0;
}
```

OUTPUT

```
5 20.5
10 20.5
```

WHY

- `b = a` makes a copy, so `b` starts as (5, 20.5).
- `b.qty = 10` changes only `b`. Variable `a` stays (5, 20.5).
- Output: first line is for `a` (5 20.5), second line for `b` (10 20.5).

A5 How Big is a Structure? MEDIUM

```c
#include <stdio.h>

struct X { int a; float b; };
struct Y { char name[10]; int age; };

int main(void) {
    printf("sizeof X = %d\n", (int)sizeof(struct X));
    printf("sizeof Y = %d\n", (int)sizeof(struct Y));
    return 0;
}
```

OUTPUT

```
sizeof X = 8
sizeof Y = 16
```

WHY

- `X`: 4 + 4 = 8, and the program prints 8. No extra bytes are needed.
- `Y`: 10 + 4 = 14, but the program prints 16. The compiler adds 2 **padding** bytes after `name` so that `age` starts at an address that is a multiple of 4.
- With `name[12]`, the members add up to 16 and the size is exactly 16, so no padding is needed. (Exact sizes can differ slightly between compilers.)

B Arrays of Structures and Functions

Handling many records and passing structures to functions.

B1 Highest Marks — EASY

```c
#include <stdio.h>

struct Student { int roll; char name[20]; float marks; };

int main(void) {
    struct Student s[10];
    int n, i, maxIndex = 0;
    float sum = 0;

    printf("How many students (max 10)?\n");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter roll, name and marks:\n");
        scanf("%d %s %f", &s[i].roll, s[i].name, &s[i].marks);
        sum += s[i].marks;
        if (s[i].marks > s[maxIndex].marks)
            maxIndex = i;
    }

    printf("Highest: %s with %.2f marks\n", s[maxIndex].name, s[maxIndex].marks);
    printf("Class average: %.2f\n", sum / n);
    return 0;
}
```

Sample input: `3 | 1 Asha 67.5 | 2 Ravi 82 | 3 Meena 74`

OUTPUT

```
How many students (max 10)?
Enter roll, name and marks:
Enter roll, name and marks:
Enter roll, name and marks:
Highest: Ravi with 82.00 marks
Class average: 74.50
```

WHY

- `maxIndex` remembers the position of the best student so far, and is updated inside the same loop that reads the data.
- The average is `sum / n`. `sum` is a `float`, so the division is not rounded down.

B2 Employee Salary Report

EASY

```c
#include <stdio.h>

struct Employee { int id; char name[20]; float salary; };

int main(void) {
    struct Employee e[5];
    int i;
    float sum = 0, avg;

    printf("Enter id, name and salary of 5 employees:\n");
    for (i = 0; i < 5; i++) {
        scanf("%d %s %f", &e[i].id, e[i].name, &e[i].salary);
        sum += e[i].salary;
    }
    avg = sum / 5;
    printf("Average salary = %.2f\n", avg);

    printf("Above average:\n");
    for (i = 0; i < 5; i++)
        if (e[i].salary > avg)
            printf("%d %s %.2f\n", e[i].id, e[i].name, e[i].salary);
    return 0;
}
```

Sample input: `1 Amit 20000 | 2 Bina 30000 | 3 Chirag 25000 | 4 Deep 40000 | 5 Esha 35000`

OUTPUT

```
Enter id, name and salary of 5 employees:
Average salary = 30000.00
Above average:
4 Deep 40000.00
5 Esha 35000.00
```

WHY

- The average must be known **before** you can compare, so two loops are needed: one to total up, one to print.
- Here the average is 30000, so only Deep and Esha are printed.

B3 Sort Students by Marks

MEDIUM

```c
#include <stdio.h>

struct Student { int roll; char name[20]; float marks; };

int main(void) {
    struct Student s[10], temp;
    int n, i, j;

    printf("How many students?\n");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d %s %f", &s[i].roll, s[i].name, &s[i].marks);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - 1 - i; j++)
            if (s[j].marks < s[j + 1].marks) {   /* descending */
                temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }

    printf("Rank list:\n");
    for (i = 0; i < n; i++)
        printf("%d. %s  %.2f\n", i + 1, s[i].name, s[i].marks);
    return 0;
}
```

Sample input: `4 | 1 Asha 67 | 2 Ravi 82 | 3 Meena 74 | 4 Karan 55`

OUTPUT

```
How many students?
Rank list:
1. Ravi  82.00
2. Meena  74.00
3. Asha  67.00
4. Karan  55.00
```

WHY

- It is the same bubble sort as for numbers, except you compare `s[j].marks` and swap whole structures.
- `<` gives descending order (highest first). Use `>` for ascending order.
- A temporary variable of the structure type is needed for the swap.

B4 Passing a Structure to a Function

EASY

```c
#include <stdio.h>

struct Student { char name[20]; int m1, m2, m3; };

float average(struct Student s) {
    return (s.m1 + s.m2 + s.m3) / 3.0;
}

void display(struct Student s) {
    printf("%s  Total: %d  Average: %.2f\n",
           s.name, s.m1 + s.m2 + s.m3, average(s));
}

int main(void) {
    struct Student st[3];
    int i;

    printf("Enter name and 3 marks for 3 students:\n");
    for (i = 0; i < 3; i++)
        scanf("%s %d %d %d", st[i].name, &st[i].m1, &st[i].m2, &st[i].m3);

    for (i = 0; i < 3; i++)
        display(st[i]);
    return 0;
}
```

Sample input: `Asha 80 70 90 | Ravi 60 65 70 | Meena 90 95 85`

OUTPUT

```
Enter name and 3 marks for 3 students:
Asha  Total: 240  Average: 80.00
Ravi  Total: 195  Average: 65.00
Meena  Total: 270  Average: 90.00
```

WHY

- The function receives a **copy** of the structure; that is enough here because it only reads values.
- Dividing by `3.0` (not `3`) keeps the decimal part of the average.
- `display` can call `average`, so the formula is written only once.

B5 A Function that Changes the Structure

MEDIUM

```c
#include <stdio.h>

struct Employee { int id; char name[20]; float salary; };

void giveBonus(struct Employee *e) {
    e->salary = e->salary + e->salary * 10 / 100;
}

void giveBonusCopy(struct Employee e) {
    e.salary = e.salary + e.salary * 10 / 100;   /* changes only the copy */
}

int main(void) {
    struct Employee emp = {1, "Ravi", 20000};

    printf("Before:            %.2f\n", emp.salary);
    giveBonus(&emp);
    printf("After giveBonus:   %.2f\n", emp.salary);
    giveBonusCopy(emp);
    printf("After copy version: %.2f\n", emp.salary);
    return 0;
}
```

OUTPUT

```
Before:            20000.00
After giveBonus:   22000.00
After copy version: 22000.00
```

WHY

- `giveBonus(&emp)` sends the **address**, so the function changes the real `emp`. Inside it, `e->salary` is the same as `(*e).salary`.
- `giveBonusCopy(emp)` receives a copy. It changes only its own copy, which disappears when the function ends, so the salary in `main` does not change.

C

Nested Structures and Pointers

A structure inside a structure, and the arrow operator.

C1 Person with Date of Birth

EASY

```c
#include <stdio.h>

struct Date { int d, m, y; };
struct Person { char name[20]; struct Date dob; };

/* returns 1 if date a is later than date b */
int isLater(struct Date a, struct Date b) {
    if (a.y != b.y) return a.y > b.y;
    if (a.m != b.m) return a.m > b.m;
    return a.d > b.d;
}

int main(void) {
    struct Person p[3];
    int i, youngest = 0;

    printf("Enter name and DOB (d m y) for 3 persons:\n");
    for (i = 0; i < 3; i++)
        scanf("%s %d %d %d", p[i].name, &p[i].dob.d, &p[i].dob.m, &p[i].dob.y);

    for (i = 0; i < 3; i++) {
        printf("Name: %s  DOB: %d/%d/%d\n",
               p[i].name, p[i].dob.d, p[i].dob.m, p[i].dob.y);
        if (isLater(p[i].dob, p[youngest].dob))
            youngest = i;
    }
    printf("Youngest: %s\n", p[youngest].name);
    return 0;
}
```

Sample input: `Asha 12 5 2004 | Ravi 3 11 2003 | Meena 25 1 2005`

OUTPUT

```
Enter name and DOB (d m y) for 3 persons:
Name: Asha  DOB: 12/5/2004
Name: Ravi  DOB: 3/11/2003
Name: Meena  DOB: 25/1/2005
Youngest: Meena
```

WHY

- Inner members need two dots: `p[i].dob.d`.
- The youngest person has the **latest** date of birth. `isLater` compares year first, then month, then day.
- Structures cannot be compared with `>`, so a small helper function is needed.

C2 Student with Marks Structure

EASY

```c
#include <stdio.h>

struct Marks { int m1, m2, m3; };
struct Student { int roll; char name[20]; struct Marks mk; };

int main(void) {
    struct Student s[3];
    int i, total;

    printf("Enter roll, name and 3 marks for 3 students:\n");
    for (i = 0; i < 3; i++)
        scanf("%d %s %d %d %d", &s[i].roll, s[i].name,
              &s[i].mk.m1, &s[i].mk.m2, &s[i].mk.m3);

    for (i = 0; i < 3; i++) {
        total = s[i].mk.m1 + s[i].mk.m2 + s[i].mk.m3;
        printf("%s  Total: %d  Percentage: %.2f\n", s[i].name, total, total / 3.0);
    }
    return 0;
}
```

Sample input: `1 Asha 80 70 90 | 2 Ravi 60 65 70 | 3 Meena 90 95 85`

OUTPUT

```
Enter roll, name and 3 marks for 3 students:
Asha  Total: 240  Percentage: 80.00
Ravi  Total: 195  Percentage: 65.00
Meena  Total: 270  Percentage: 90.00
```

WHY

- Marks are inside `mk`, which is inside the student, so the full path is `s[i].mk.m1`.
- Percentage = total / 3.0 when each subject is out of 100.

C3 Predict the Output: Pointer to Structure

EASY

```c
#include <stdio.h>

struct Rect { int w, h; };

int main(void) {
    struct Rect r = {4, 5};
    struct Rect *p = &r;
    p->w = 10;
    printf("%d\n", r.w);
    printf("%d\n", (*p).h);
    printf("%d\n", p->w * p->h);
    return 0;
}
```

OUTPUT

```
10
5
50
```

WHY

- `p` holds the address of `r`, so `p->w = 10` changes `r.w` itself. That is why the first line prints 10.
- `(*p).h` is the same as `p->h`, which is 5. The last line is 10 * 5 = 50.
- `*p.w` is read as `*(p.w)` because the dot has higher priority than `*`. `p` is a pointer, not a structure, so `p.w` is an error. The brackets in `(*p).w` fix the order.

C4 Walking Through an Array with a Pointer

MEDIUM

```c
#include <stdio.h>

struct Point { int x, y; };

int main(void) {
    struct Point pt[3] = {{1, 2}, {3, 4}, {5, 6}};
    struct Point *p = pt;
    int i;

    printf("Using p++:\n");
    for (i = 0; i < 3; i++) {
        printf("(%d, %d)\n", p->x, p->y);
        p++;
    }

    p = pt;                       /* go back to the start */
    printf("Using (p + i):\n");
    for (i = 0; i < 3; i++)
        printf("(%d, %d)\n", (p + i)->x, (p + i)->y);
    return 0;
}
```

OUTPUT

```
Using p++:
(1, 2)
(3, 4)
(5, 6)
Using (p + i):
(1, 2)
(3, 4)
(5, 6)
```

WHY

- `p = pt;` needs no `&` because the array name is already the address of the first element.
- `p++` moves to the next structure. `(p + i)->x` reaches element `i` without moving `p`.
- Reset with `p = pt;` before the second loop, because the first loop has moved `p` past the end.

D Unions

Shared memory, sizeof, and when to use a union.

D1 Your First Union

EASY

```c
#include <stdio.h>

union Data { int i; float f; char c; };

int main(void) {
    union Data d;

    d.i = 10;
    printf("i = %d\n", d.i);

    d.f = 3.5;
    printf("f = %f\n", d.f);
    printf("i = %d\n", d.i);      /* unexpected value */
    return 0;
}
```

OUTPUT

```
i = 10
f = 3.500000
i = 1080033280
```

WHY

- All members share one memory block, so storing `d.f` overwrites the bytes that held `d.i`.
- Reading `d.i` now shows the bits of the float 3.5 interpreted as an integer. The exact number can differ between machines; it is not a meaningful value.
- Rule: only the member stored **last** is safe to read.

D2 sizeof: Structure vs Union

EASY

```c
#include <stdio.h>

struct S { int i; float f; char c; };
union  U { int i; float f; char c; };

int main(void) {
    printf("sizeof(struct S) = %d\n", (int)sizeof(struct S));
    printf("sizeof(union U)  = %d\n", (int)sizeof(union U));
    return 0;
}
```

OUTPUT

```
sizeof(struct S) = 12
sizeof(union U)  = 4
```

WHY

- The structure keeps separate space for each member: 4 + 4 + 1 = 9, rounded up to 12 because of padding.
- The union needs space only for its largest member (`int` or `float`, 4 bytes), because all members share it.

D3 Predict the Output: Union

EASY

```c
#include <stdio.h>

union Num { int a; int b; };

int main(void) {
    union Num n;
    n.a = 25;
    printf("%d\n", n.b);
    n.b = 40;
    printf("%d\n", n.a);
    return 0;
}
```

OUTPUT

```
25
40
```

WHY

- `a` and `b` are two names for the same memory.
- After `n.a = 25`, reading `n.b` gives 25. After `n.b = 40`, reading `n.a` gives 40.

D4 Product Code: Number or Text

MEDIUM

```c
#include <stdio.h>

union Code { int num; char text[10]; };

int main(void) {
    union Code c;
    int type;

    printf("Type (1 = number, 2 = text):\n");
    scanf("%d", &type);

    printf("Enter the code:\n");
    if (type == 1)
        scanf("%d", &c.num);
    else
        scanf("%s", c.text);

    if (type == 1)
        printf("Product code (number): %d\n", c.num);
    else
        printf("Product code (text): %s\n", c.text);
    return 0;
}
```

Sample input: 2 | AB12

OUTPUT

```
Type (1 = number, 2 = text):
Enter the code:
Product code (text): AB12
```

WHY

- A union cannot tell which member was stored last, so the program keeps its own flag, `type`.
- Check `type` before reading: `%d` for `c.num`, `%s` for `c.text`.
- The text can hold up to 9 characters (the 10th byte is for the end marker).

D5 Structure vs Union: Compare

CONCEPT

Point	Structure	Union
Keyword	struct	union
Memory	Every member has its own memory	All members share one memory block
sizeof	About the sum of all members (plus padding)	Size of the largest member
Valid values at one time	All members	Only the last stored member

WHY

- **Example for a structure:** a student record. Roll number, name and marks are all needed at the same time, so each needs its own space.
- **Example for a union:** the product code in D4. A code is either a number or a text, never both, so one shared block is enough and memory is saved.

E Mixed Practice

Spot the errors, then put it all together.

E1 Find the Error

EASY

```
/* (a) */ struct S { int x; float y; };      /* semicolon added */
           struct S s1;

/* (b) */ #include <string.h>
           struct Student { char name[20]; } st;
           strcpy(st.name, "Ravi");

/* (c) */ struct S a = {1, 2.5}, b = {1, 2.5};
           if (a.x == b.x && a.y == b.y)
               printf("Same");

/* (d) */ struct S { int x; };
           struct S s1;                       /* 'struct' added */
```

WHY

- **(a)** A structure definition must end with a semicolon after the closing brace.
- **(b)** A character array cannot be assigned with `=`. Use `strcpy` and add `#include <string.h>`.
- **(c)** Structures cannot be compared with `==`. Compare each member.
- **(d)** In C you must write the word `struct` when declaring the variable (or use `typedef` to avoid it).

E2 Mini Library Menu

MEDIUM

```c
#include <stdio.h>

struct Book { int id; char title[30]; float price; };

int main(void) {
    struct Book b[5];
    int count = 0, choice, i, id, found;

    do {
        printf("\n1. Add  2. Display  3. Search  4. Exit\n");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            if (count == 5) {
                printf("Library is full\n");
                break;
            }
            printf("Enter id, title and price:\n");
            scanf("%d %s %f", &b[count].id, b[count].title, &b[count].price);
            count++;
            break;
        case 2:
            for (i = 0; i < count; i++)
                printf("%d  %s  %.2f\n", b[i].id, b[i].title, b[i].price);
            break;
        case 3:
            printf("Enter id to search:\n");
            scanf("%d", &id);
            found = 0;
            for (i = 0; i < count; i++)
                if (b[i].id == id) {
                    printf("Found: %s  %.2f\n", b[i].title, b[i].price);
                    found = 1;
                }
            if (!found)
                printf("Not found\n");
            break;
        case 4:
            printf("Bye\n");
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 4);
    return 0;
}
```

Sample input: `1 | 101 Python 450 | 1 | 102 Java 520 | 2 | 3 | 102 | 3 | 999 | 4`

OUTPUT

```
1. Add  2. Display  3. Search  4. Exit
Enter id, title and price:

1. Add  2. Display  3. Search  4. Exit
Enter id, title and price:

1. Add  2. Display  3. Search  4. Exit
101  Python  450.00
102  Java  520.00

1. Add  2. Display  3. Search  4. Exit
Enter id to search:
Found: Java  520.00

1. Add  2. Display  3. Search  4. Exit
Enter id to search:
Not found

1. Add  2. Display  3. Search  4. Exit
Bye
```

WHY

- `count` tracks how many books are stored. A new book goes into `b[count]`, then `count++`.
- The `found` flag starts at 0 and becomes 1 only if a matching id is seen, so "Not found" is printed correctly.
- `do ... while` makes the menu appear at least once and repeats until the user chooses 4.

E3 Result Card

MEDIUM

```c
#include <stdio.h>

struct Marks { int m1, m2, m3; };
struct Student {
    int roll;
    char name[20];
    struct Marks mk;
    float percent;
    char grade;
};

int main(void) {
    struct Student s[5];
    int n, i, top = 0;

    printf("How many students (max 5)?\n");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        scanf("%d %s %d %d %d", &s[i].roll, s[i].name,
              &s[i].mk.m1, &s[i].mk.m2, &s[i].mk.m3);

        s[i].percent = (s[i].mk.m1 + s[i].mk.m2 + s[i].mk.m3) / 3.0;

        if (s[i].percent >= 75)      s[i].grade = 'A';
        else if (s[i].percent >= 60) s[i].grade = 'B';
        else if (s[i].percent >= 40) s[i].grade = 'C';
        else                         s[i].grade = 'F';

        if (s[i].percent > s[top].percent)
            top = i;
    }

    printf("\nRoll  Name     Percent  Grade\n");
    for (i = 0; i < n; i++)
        printf("%-5d %-8s %7.2f  %c\n",
               s[i].roll, s[i].name, s[i].percent, s[i].grade);

    printf("\nTopper: %s (%.2f%%)\n", s[top].name, s[top].percent);
    return 0;
}
```

Sample input: `3 | 1 Asha 80 70 90 | 2 Ravi 60 65 70 | 3 Meena 90 95 85`

OUTPUT

```
How many students (max 5)?

Roll  Name     Percent  Grade
1     Asha       80.00  A
2     Ravi       65.00  B
3     Meena      90.00  A

Topper: Meena (90.00%)
```

WHY

- Work in steps: read, calculate percentage and grade, then print. Percent and grade are stored **inside** the structure.
- Grades use `if ... else if`, checking from the highest limit downwards.
- The topper is found the same way as the highest marks in B1.